

CSE 2216 - Application Development Lab Project Report



Murir Tin

Team Dev_Mavericks

Submitted By

Sumaiya Rahman Soma (07)

MD. Ishrak Faisal (12)

Jobaer Hossain Tamim (24)

Chowdhury Shafahid Rahman (55)

29th Batch

Department of Computer Science & Engineering
University of Dhaka

Submitted On

13 July 2025

Table of Contents

1. Introduction	3
1.1 About the App	3
1.2 Motivation	3
1.3 Features (Bullet Points)	3
1.4 Tools, Technologies, and Frameworks Used	4
1.5 Individual Responsibilities	4
1.4.1 Member 1	4
1.4.2 Member 2	
1.4.3 Member 3	5
1.4.4 Member 4	6
2. Design and Implementation	8
2.1 UI Developed (Screenshots)	8
2.2 Database Schema	16
2.3 Code Repository Link	17
2.4 Video Demo Link	17
3. Conclusion	19
3.1 Challenges and Solutions	19
3.2 Lessons Learned	26
3.3 Future Plan	26

1. Introduction

1.1 About the App

‘Murir Tin’ is a smart local bus app that uniquely combines color-coded routes, live GPS tracking, QR-based easy ticket booking, a complaint box to submit, track, and view complaints and emergency features — making daily transportation safer, smoother, and easier.

1.2 Motivation

Public transport in Dhaka is often confusing, crowded, and unsafe. Many people struggle with finding the right bus, waiting for long hours, and being overcharged by conductors. For daily commuters like garment workers, elderly visitors, students, and office goers, these small problems add up and make travel stressful.

The government’s plan to launch a color-coded bus system is a big step toward solving this. It aims to reduce the number of routes and make the system more organized. However, passengers will still need support to use it easily. Just having colored buses is not enough unless people know which one to take, where it is, how full it is, and how to travel safely.

To support this change and improve the daily commute experience, the **Murir Tin** app has been developed. It is built to make public transport easier and safer for everyone. The app helps users find nearby routes, view live bus locations, book tickets using QR codes or manual codes, and report complaints or emergencies when needed. All features are designed to match real-world needs and are accessible to people with varying levels of tech experience.

Murir Tin aims to reduce the stress of daily travel and make public transport in Dhaka more reliable, organized, and user-friendly.

1.3 Features (Bullet Points)

1. Find Bus

- Shows the user’s current location on the map.
- Displays all bus stoppages and color-coded routes.
- Highlights the nearest bus stoppages for easier access.

2.Live Map

- Shows real-time locations of available buses.

- Helps users know when and where to catch the next bus.

3. Book Ticket

- Calculates fare based on distance (per kilometer pricing).
- Supports online payment through a secure gateway.
- Generates QR code tickets for contactless use.
- Offers PDF versions of tickets for offline use or printing.

4. Complaint Box

- Allows users to write complaints linked to a specific bus or ticket.
- Shows complaint status to keep users updated.
- Displays other users' complaints, with options to like, dislike, and leave feedback.

5. Emergency SOS

- Lets users quickly send alerts in emergencies like harassment, accidents, or health issues.
- Directly connects with the nearest help center for fast assistance.

6. Profile Section

- Users can view and update their personal information.

1.4 Tools, Technologies, and Frameworks Used

1. **Framework:** Flutter
2. **Backend:** FastAPI + Supabase
3. **Database & Authentication:** Supabase
4. **Map & Location Services:** Mapbox API
5. **Version control :** github
6. **Backend Hosting :** Vercel

1.5 Individual Responsibilities

1.4.1 Team Member 1: Sumaiya Rahman Soma (SK-07)

Responsibility: Frontend

- Designed the app logo.
- Built the landing page to fetch user details, get live location, and display the current address.
- Created sign-up and log-in user interfaces.
- Designed the Booked Ticket page UI with a QR scanner placeholder for future scanning.
- Developed the SOS screen with an emergency call feature.

- Handled runtime permissions for location services.
- Built the Booking History page with data sorting, PDF generation, and a print option for offline use.
- Created the About Us page with app and team member details.
- Applied asynchronous programming to handle location fetching and user data display smoothly.
- Created the database structure for booking tickets and booking history, including natural joins for linking user and ticket data.
- Version Control (Git / Github).
- Combined the initial basic structure of the codebase

1.4.2 Team Member 2: Mir Md. Ishrak Faisal (FH-12)

Responsibility: Frontend

- Designed the Complaint Box feature, allowing users to select a bus company and submit complaint titles and details through the app.
- Built the Complaint Status Page featuring a button on each complaint card that navigates to a detailed view; the page stores all complaints along with their time, date, and user-submitted title.
- Developed the Current Status page UI where users can check if their complaint is Submitted, Accepted, or Solved.
- Created a social platform within the app where users can view all complaints with anonymous profile pictures, use See More/Show Less options, react with Like buttons, and filter complaints using a search bar.
- Designed My Profile page for all informations including profile picture of a user.
- Built the Edit Profile screen to allow users to update their information and change their profile picture.
- Developed user interface of Settings for managing password.
- Version Control (Git & Github)

1.4.3 Team Member 3: Jobaer Hossain Tamim (SH-24)

Responsibility: Map & Backend

- Design:
 - Initially designed Figma Prototype (Later the color, logo and design were revised by the team to a great extent.) .
 - Improved the whole UI and gave it a user friendly overview.

- Frontend :
 - Styling map.
 - Handled user-friendly error messages.
 - Handled location permission(used permission_handler).
 - Showing user location.
 - Showing walking path for nearest stoppage.
 - Animated drawer to show all stoppages of a route.
 - Live bus feature
 - Updating the Social Issues UI
 - Updating Booking Ticket flow(latest one)
- Backend :
 - Api endpoints for getting all routes, bus stops.
 - Tried to use something like MVC architecture (though could not fully maintain that due to incapability and lack of experience).
 - Fetching the nearest stoppage data from the mapbox api.
 - Workable react system.
- Database :
 - Creating a database for bus stop location from life data

1.4.4 Team Member 4: Chowdhury Shafahid Rahman

Responsibility: Backend & Payment Gateway

- API development with FastAPI:
 - Designed and implemented RESTful endpoints to handle various backend functionalities of the app. This included setting up endpoints for user data retrieval, ticket booking, complaint submission, edit profile etc.
- Database Management with Supabase:
 - Designed and created the database schema in Supabase, including defining tables such as user_profiles, complaints, bus_company, etc., along with their relationships and constraints.
 - Integrated FastAPI with Supabase (PostgreSQL) using the Supabase Python client.
 - Handled CRUD operations on user-related tables (e.g., user_profile, complaints, etc.) while ensuring data consistency.
- Authentication and Authorization:
 - Built token verification mechanisms to authenticate users via JWT tokens obtained from Supabase Auth.
 - Ensured that protected endpoints were only accessible by authorized users.

- Error Handling and Data Validation:
 - Implemented Pydantic models for request and response validation.
 - Handled edge cases with appropriate HTTP status codes and descriptive error messages.
- Payment Gateway Integration:
 - Integrated the bKash Payment Gateway in sandbox mode using the Flutter bKash package to simulate payment transactions within the app.
- Testing and Debugging:
 - Used FastAPI's built-in Swagger documentation interface (/docs) to test endpoints, verify request/response formats, and debug API behaviors.
 - Assisted frontend integration by debugging token handling and data mismatch issues.

2. Design and Implementation

2.1 UI Developed (Screenshots)

1) Splash Screen:

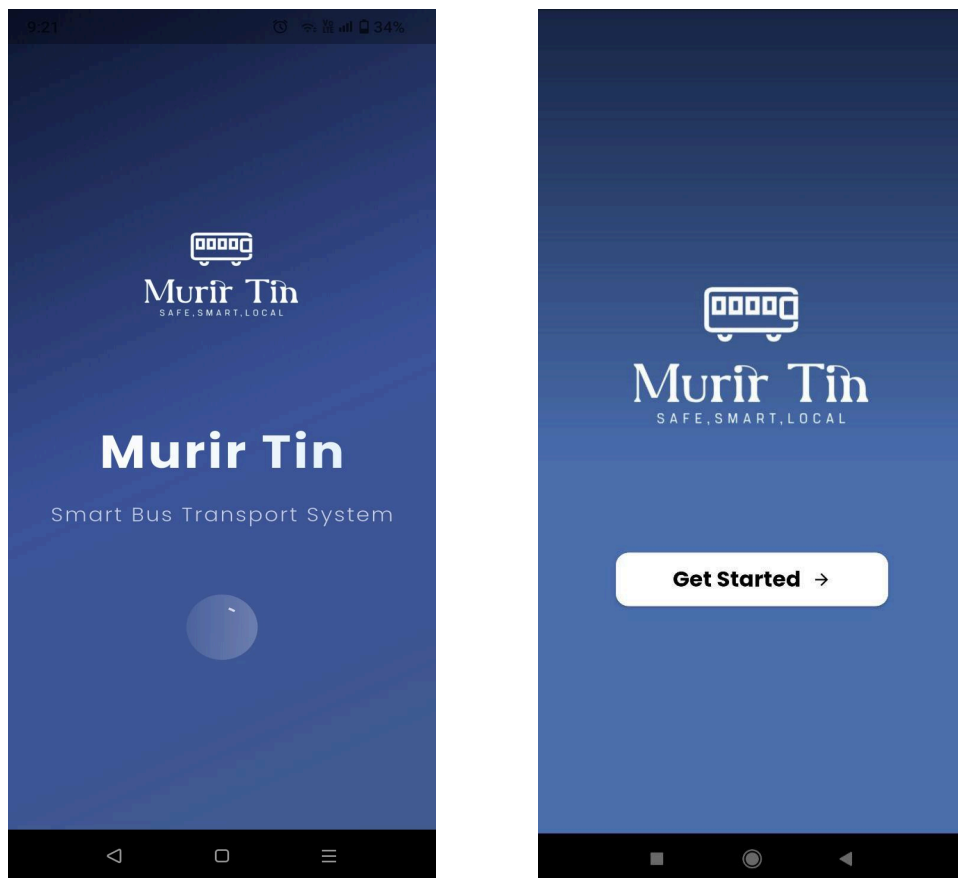


Fig: Splash Screen

2) SignUp and SignIn Screen:

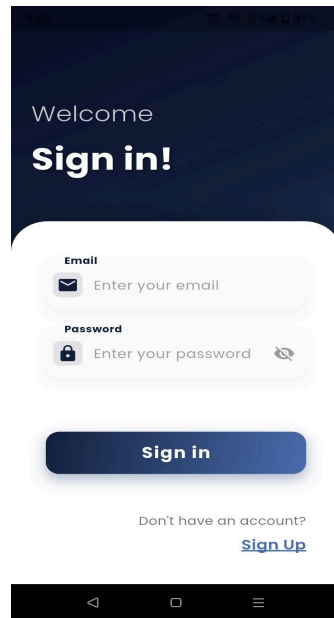


Fig: SignIn screen

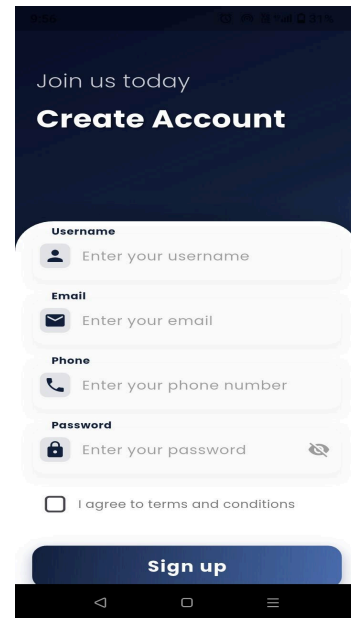


Fig: SignUp screen

3) Landing Page:

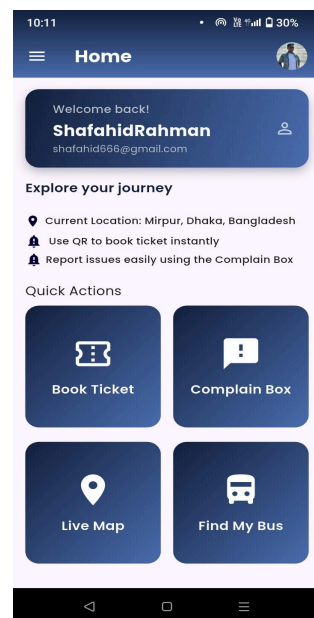


Fig: Landing screen

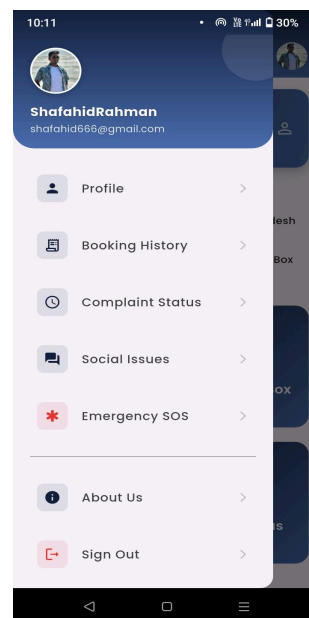


Fig: SideBar

4) Book Ticket:

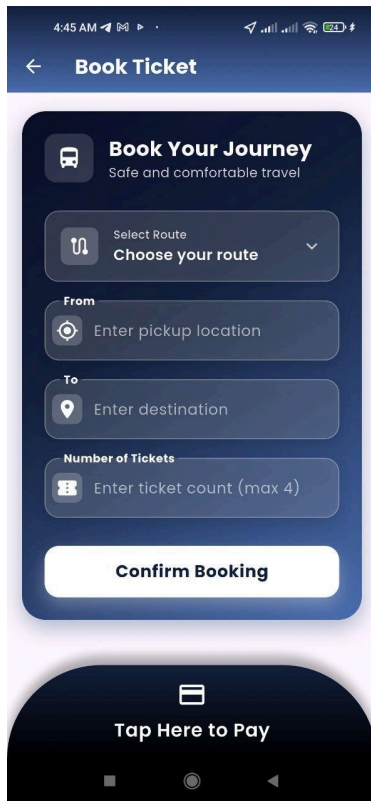


Fig: Route Selection

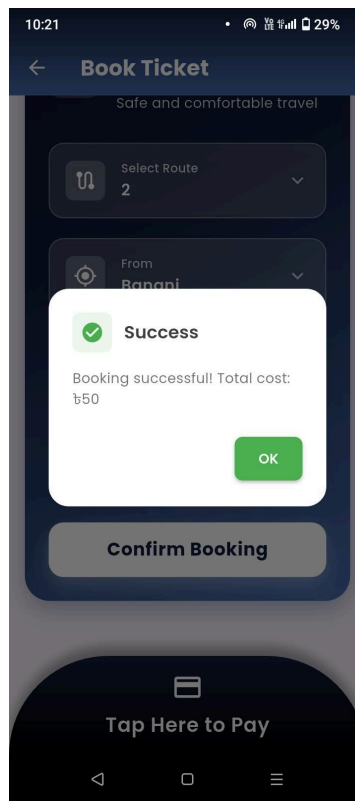


Fig: Total Cost Popup

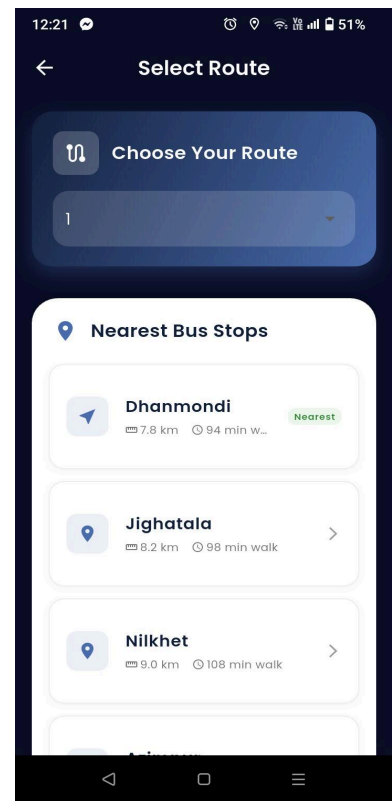


Fig: Nearest Route

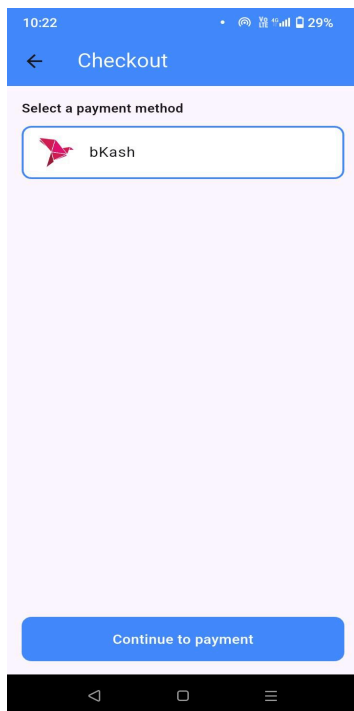


Fig: Payment Gateway

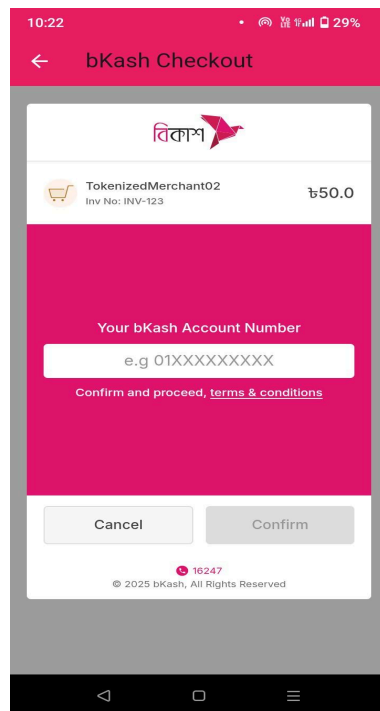


Fig: Payment Screen

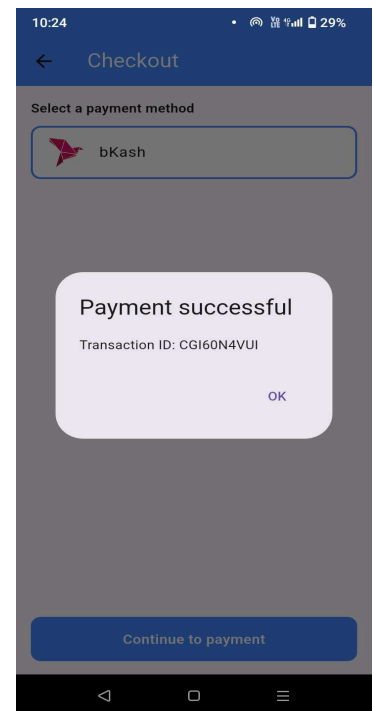


Fig: Payment Success

5) Ticket Book History:

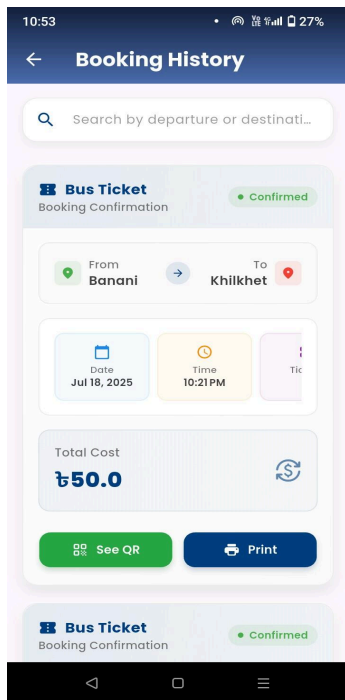


Fig: Book History Screen

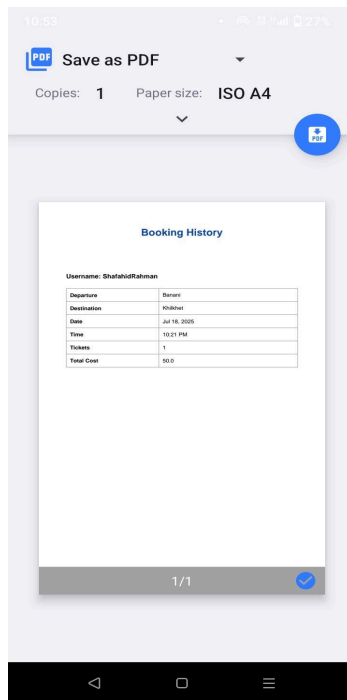


Fig: Ticket PDF

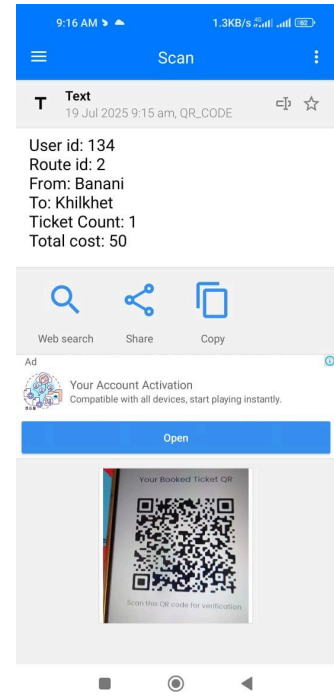


Fig: Scanned QR code

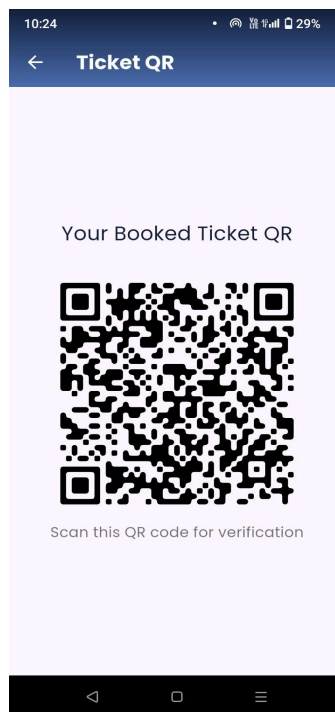


Fig: Generated QR code

6) Complaint Box:

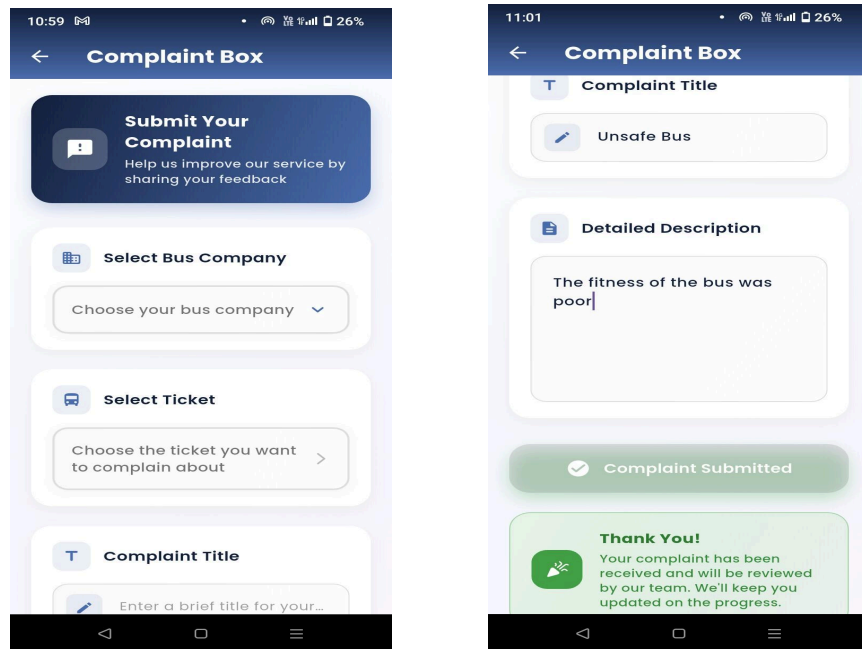


Fig: Complaint Box Screen

7) Complain Status:

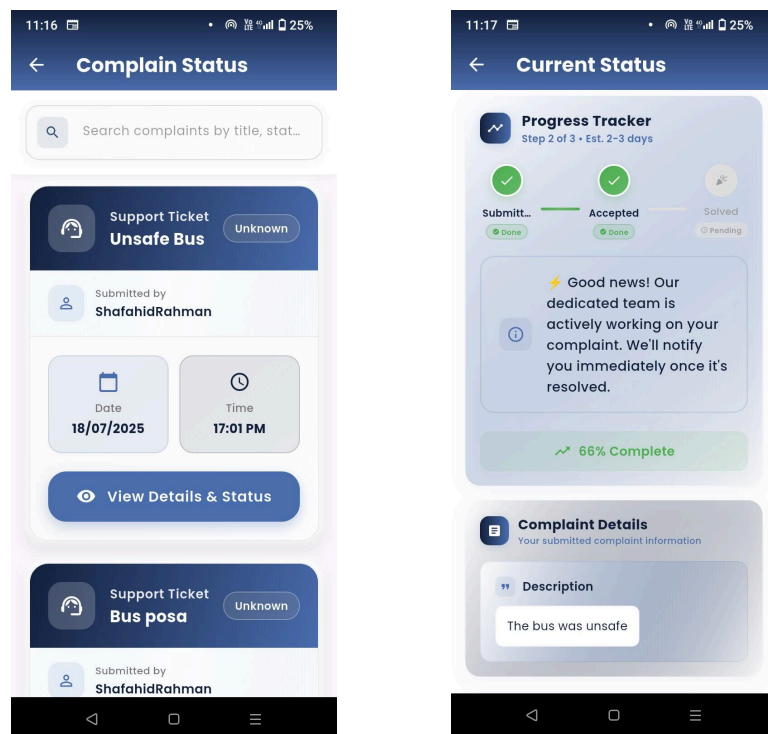


Fig: Complaint Status

8) Social Issues:

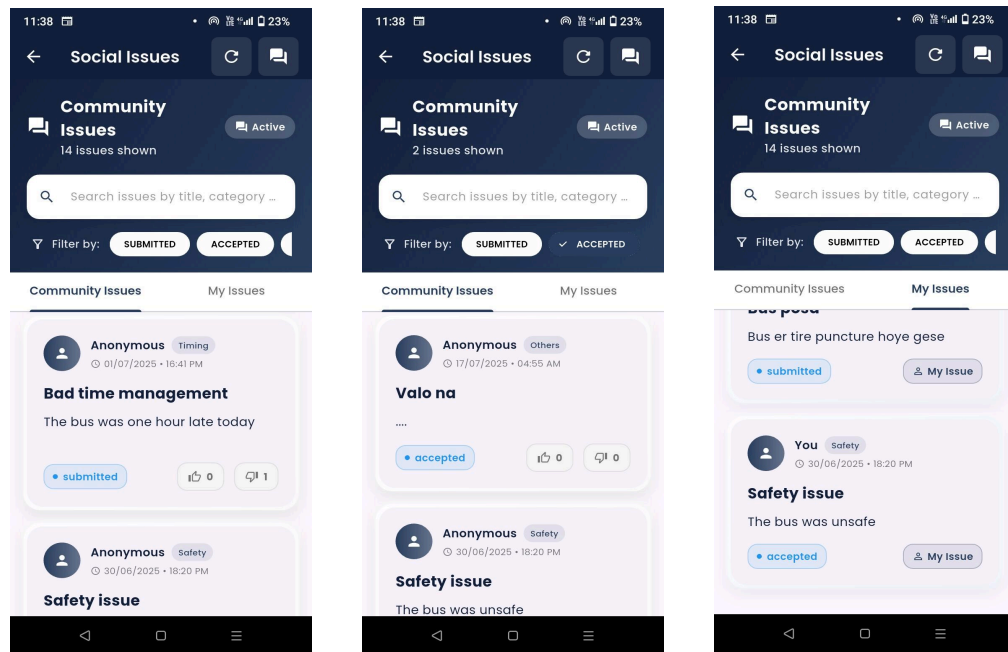


Fig: Social Issues Screen with Search Bar

9) SOS Button:

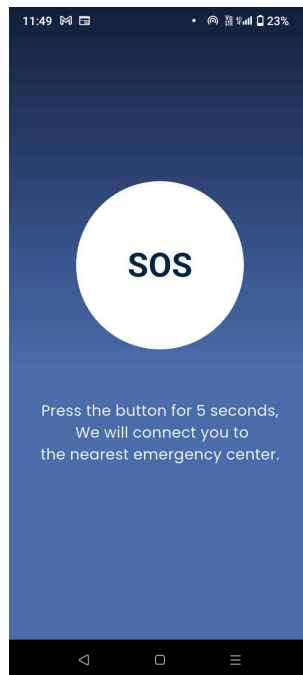


Fig: SOS Button Screen

10) Find My Bus:

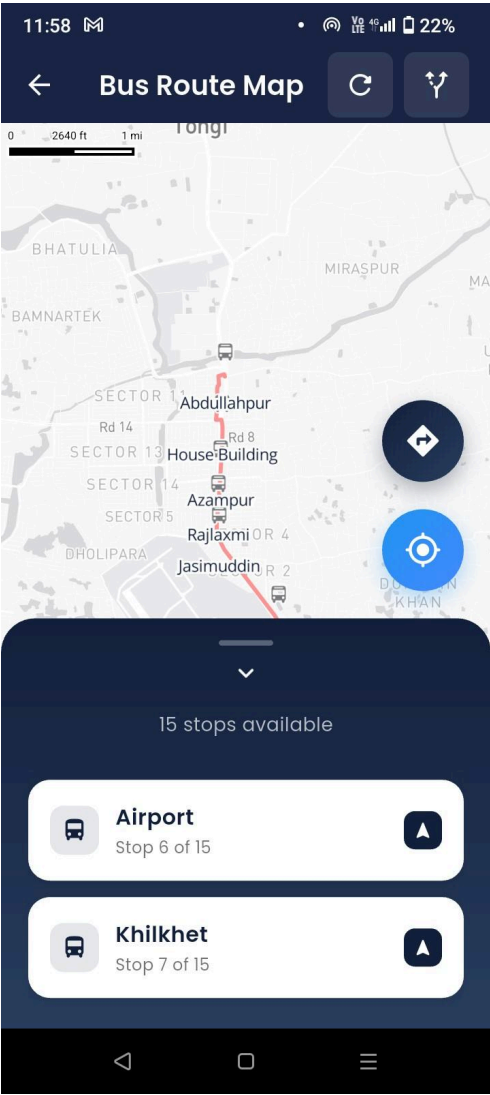


Fig: Bus Route Map

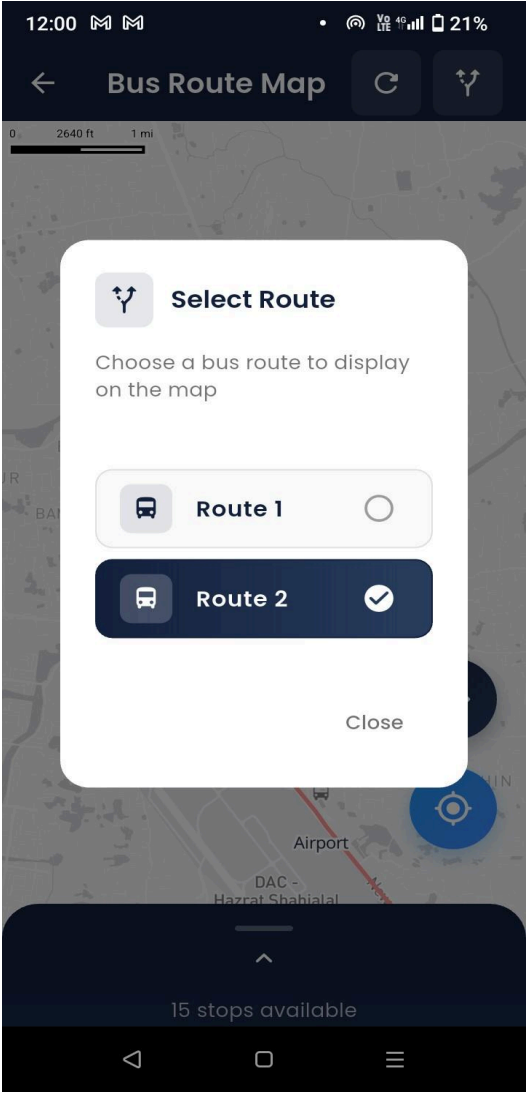


Fig: Route Selection

11) Profile Section:

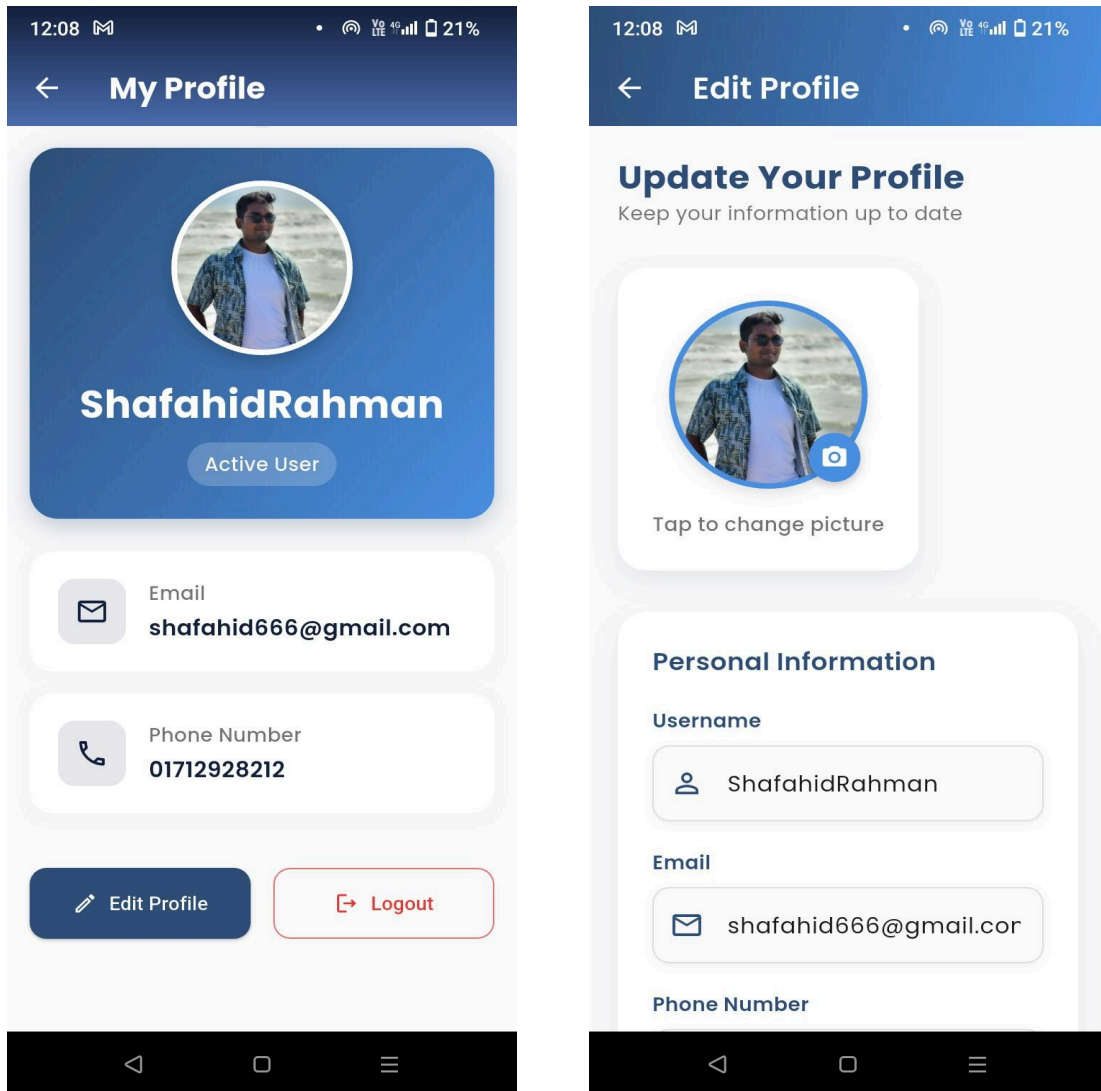


Fig: User Profile & Edit Profile

12) Live Bus Track:

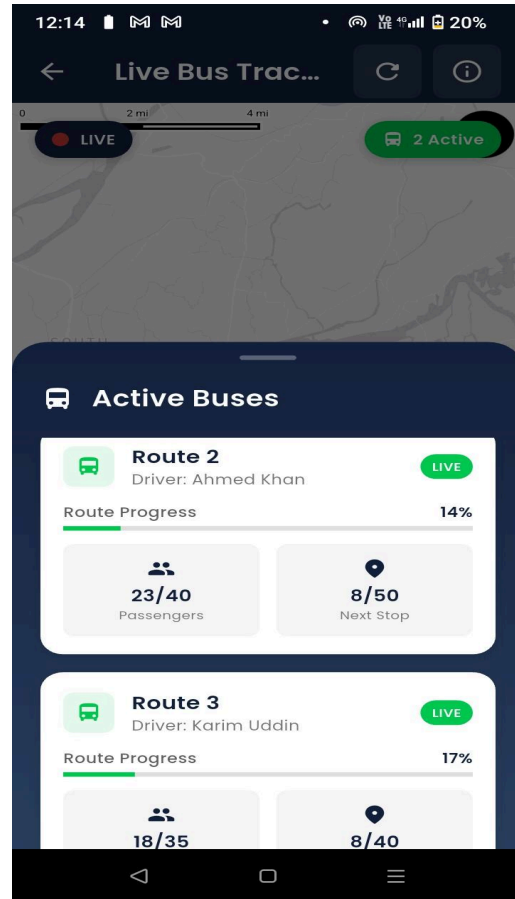
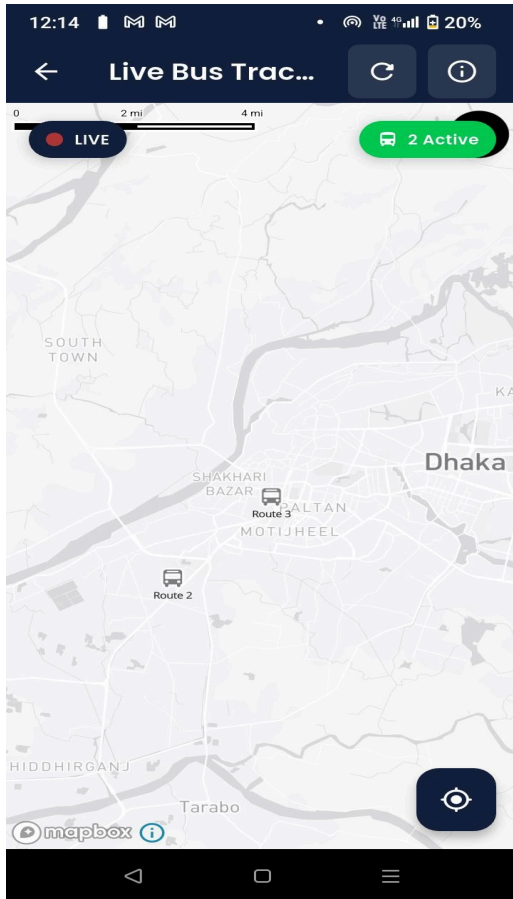


Fig: Live Bus Track Using Map

13) About Us Page:

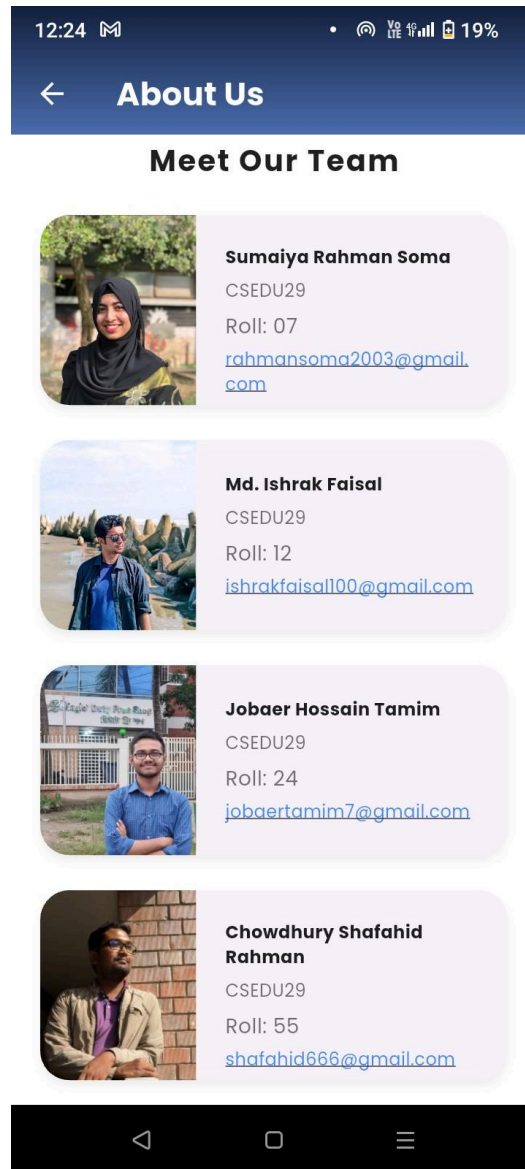
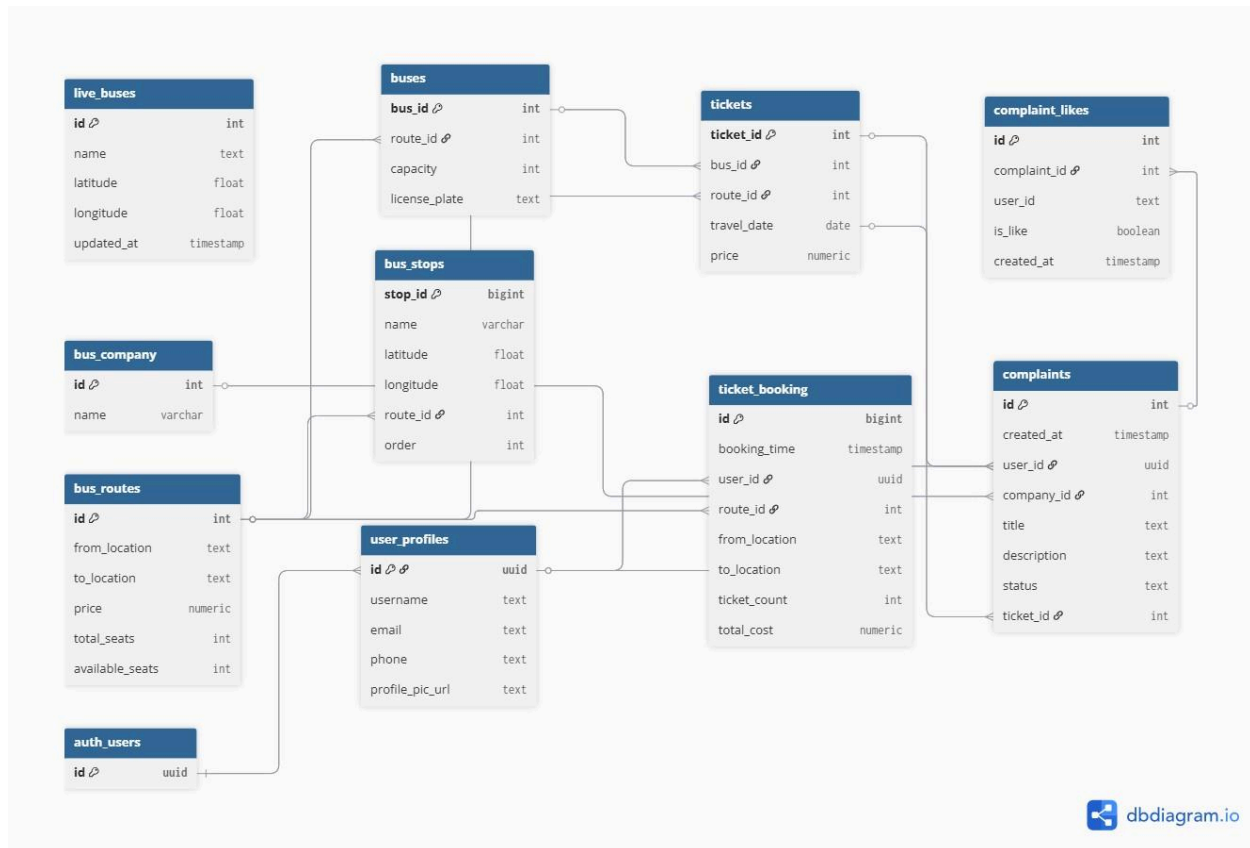


Fig: About Us Screen

2.2 Database Schema



2.3 Code Repository Link

Frontend Github Link: [srs4929/Murir_Tin_Frontend](https://github.com/srs4929/Murir_Tin_Frontend)

Note: The apk is in the release of this repository.

Backend Github Repository Link : [MurirTinBackend](https://github.com/MurirTinBackend)

2.4 Video Demo Link

YouTube Link: [▶ Murir Tin – Smart Public Transport App | CSE-2216 -Mobile App Developm...](#)

Special Consideration: Upon advice we have updated the book ticket UI/UX flow, filter system in social issues and sorted order in book ticket system.

3. Conclusion

3.1 Challenges and Solutions

Challenge 1: NDK Version Mismatch

Issue:

While setting up the Flutter project, we encountered an issue with the Android Native Development Kit (NDK). The installed NDK version did not match the version required by some dependencies, causing build errors and failed Gradle syncs during development.

Impact:

This mismatch blocked us from building and testing the app on Android emulators and physical devices, creating unexpected difficulties during development and testing phases. It also made it difficult to verify new features quickly on real devices.

Solution:

We resolved this by explicitly specifying the compatible NDK version in our `local.properties` file and aligning our Flutter and Gradle versions with official requirements. We also cross-checked the recommended NDK version in Android Studio's SDK Manager to maintain consistency across team members' environments.

Challenge 2: URL Launcher Configuration Issue

Issue:

We faced a problem when adding the `url_launcher` plugin to open external links. At first, we relied on LLM suggestions, but those didn't include the latest Android requirements.

Because of that, we missed adding the necessary **<queries>** block in our **AndroidManifest.xml** file. For Android 11 (API level 30) and above, this part is now mandatory for the app to check for external apps safely.

Impact:

Without this our app sometimes failed to open external URLs — especially for actions like sending emails or sharing text. This broke parts of the user support flow during development. The links were not opened smoothly, which made the experience frustrating and slowed down testing too.

Solution:

We double-checked the official **url_launcher** docs and Android developer guidelines. Then, we updated our **AndroidManifest.xml** and added the **<queries>** section.

Challenge 3: Pinpointing Current Address Using Geolocator

Issue:

We wanted to show the user's current address by getting their exact GPS location using the Geolocator plugin. At first, we tried to get the precise address for each user's location.

However, in some areas, especially with poor GPS signals or less mapped regions, the app couldn't find an exact address. This caused blank or incorrect address information to appear.

Impact:

This caused inconsistencies in the app's location-based features during development.

Solution:

To fix this, we decided to implement a fallback system: if the exact address wasn't found, the app shows the nearest available address instead.

This way, even if the pinpoint location isn't perfect, users still get a useful approximate address to proceed with. It improved overall user experience and reliability.

Challenge 4: PDF Download and Print Preview

Issue:

We initially aimed to implement a Chrome-like PDF download experience within the app. For doing so, we followed advice from an LLM, but it didn't work out.

The method kept failing during development.

Impact:

Because of this, we couldn't get the PDF download feature ready on time. It slowed us down.

Solution:

Instead of relying on the LLM and facing multiple errors, we went back to YouTube tutorials and official documentation to understand how to do it properly. We then switched to saving PDFs in local storage, which worked better on different devices. We also added a print preview feature so users can see their tickets before printing.

Challenge 5: Mapbox integration issue**Issue:**

As per suggestion of ChatGPT we tried to use `mapbox_gl` for the mapbox integration. But we faced the integration error.

** What went wrong:*

A problem occurred evaluating project ':mapbox_gl'.

> SDK Registry token is null. See README.md for more information.

Impact:

It took us about one day to resolve the issue. We just needed to abandon this package. Before abandoning it we also tried several LLM (deepseek, claude, grok, qwen) to find the cause. But the error remains as we go on.

Solution:

The journey is pretty interesting for me. At first I needed my resolve to abandon the LLM support completely. Later I went to the official document of the `mapbox_gl` flutter in pub.dev. There I got an important catch. I am just quoting the line here *"If not properly configured the*

following error will be given :" So the error is given because the key is not properly set up. Okay fine then ! Let's dive into the key rescue mission ! Before this line there was a link for setting up mapbox_gl for Android. I went to the link and found that there are several steps to set up the key. I followed those steps. But in step 3 I got stuck as there is no file in that name in my flutter project. So I could not proceed with the steps. Then I went to github issues as there are some fine answers about the development. One of the issue posts piqued my interest. [HERE](#) is the reference of the issue. From the comment I realized mapbox_gl is not for flutter actually. We can use it for native android or iOS development, not for cross platform. Also noticed another thing: the mapbox_gl pub dev package was updated 3 years ago. So naturally it should have some issue with the current flutter. Then we just abandoned it and moved to an alternative package named "mapbox_map_flutter". I used the documentation and the process was real quick.

Challenge 6: Windows firewall issue

Issue:

At first we used localhost for the backend and frontend connections. But an important issue arises then. How to connect the wireless debug mode? We then just switched to adb for wireless connection. But the localhost didn't work as expected. The solution was pretty straightforward. We just need to give 0.0.0.0 to get in all the devices. But after that it still did not work.

Impact:

Well it did cost several hours (4-5) hours to fix the issue. No major impact except debugging time cost here.

Solution:

At first we checked the same code on the PC using localhost. The code worked just fine in chrome. So the issue was not with the frontend it must be the backend. Then we used the thunderbolt client (api tester like postman) in VScode. There it was just fine. (Note: for checking whether the server is running we simply put a message "Server is running at full throttle" in the root of the api. The api was working nicely on the computer but it was not working on other devices (Laptop and Mobile phone). Now having lost, we just went to chatGPT to find the error. It gave us the answer to this [post](#). It simply told us to open the 8000 port through the

firewall. It did work for us. But for a better one we shifted to macOS (it wants permission before exposing the port).

Challenge 7: Adding style layer in Map

Issue:

The map was showing just fine. But the marker annotation and route was not showing in the map.

Impact:

As I worked mainly on the map, it did hamper the future progress of map integration and styling. Well from my perspective this is a learning flow rather than a challenge.

Solution:

Just went to mapbox flutter sdk documentation for the solution. There indeed I found a process to use. But I personally got disappointed as the solution is not too detailed. So as usual I went to chatGPT to understand the code part. But the first time I didn't get an answer (might be my ignorance or less understanding). I then used the gpt search tool hoping to get a better result. When I did so I found an interesting website. It's name is [deepwiki](#). From what I saw it is an AI powered website for documenting the github repositories. I found some interesting insights about how the package works, its architecture and many more. I am pretty much interested in this website. But it is a shame that I could not deep dive into the website. But I'm gonna to explore this website more in the future.

Challenge 8: Live Location

Issue:

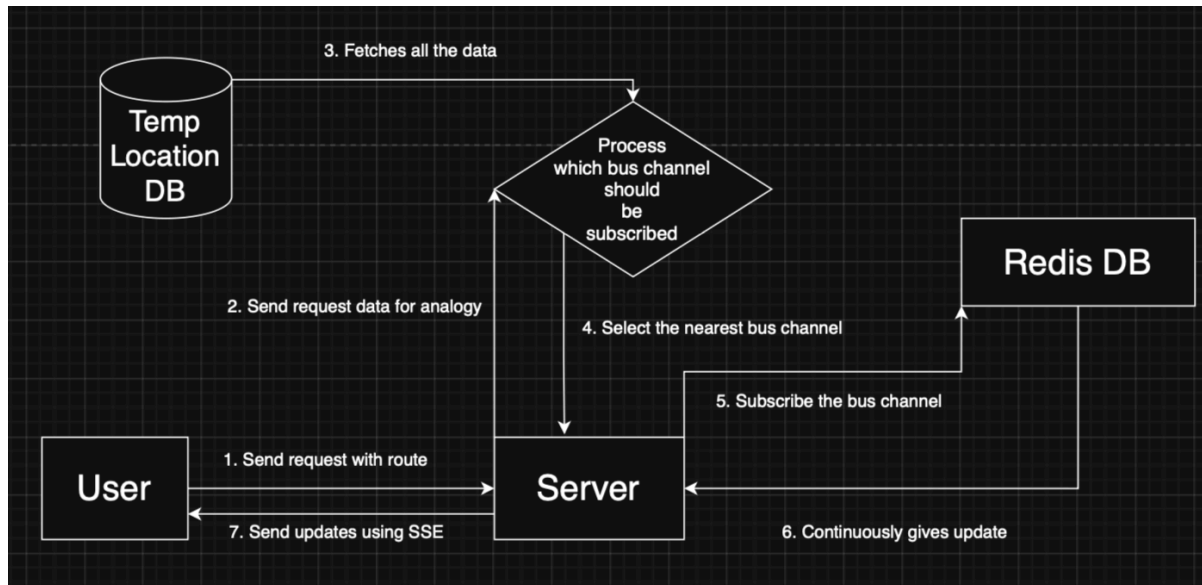
Struggling to how to send the live location using a consistent channel

Impact:

To reduce this impact mainly I am trying parallelly doing the jobs. I got the solution but guess this would take time to implement.

Solution:

Below is a diagram to solve this issue : (Courtesy [draw.io](#))



Though I am working on this architecture I am not confident about finishing this on time. For now just 1, 5, 6 and 7 works just fine. We still need to work on the analogy and UI representation.

Challenge 9: Delayed Code Integration

Issue:

We followed a branching model where each member worked mostly in separate branches, but we did not merge changes back to the main branch frequently. This approach helped avoid immediate conflicts but created uncertainty and larger conflicts when we needed to integrate everything together.

Impact:

Because branches diverged, we faced difficulties when we needed to test features together. Some features depended on other unfinished parts, which delayed full integration and slowed testing on real devices. It also made debugging more difficult because unmerged code was harder to test end-to-end.

Solution:

We realized we needed more frequent check-ins and earlier merging of smaller parts. We are now planning to finish isolated parts first, then integrate them step by step to reduce merge risks. In the future, we will adopt a clearer **feature branch** → **pull request** → **review** → **merge** workflow, to catch issues early and test integrated features sooner.

Challenge 10: Supabase Logic & QR Scanner Data Sync

Issue:

While implementing the ticket booking and validation system, we faced a problem with updating the total ticket after booking in Supabase. When a ticket was “cut” (used), the system was supposed to reduce the total cost in the database. However, the update did not work as expected in Supabase; the value stayed the same.

Interestingly, we tested the same logic with a FastAPI backend, and it worked perfectly, so the issue was likely with Supabase’s client-side or server-side triggers or how we handled updates.

Additionally, when using the QR scanner to validate tickets, we ran into trouble sending the scanned data reliably back to the backend. This caused mismatches between the scanned ticket and the database state.

Impact:

This caused inconsistencies in the ticket booking history and total cost calculations, which could confuse both users and admins. It also slowed down development because the same logic needed to be tested in multiple backend approaches.

Solution:

We decided to handle this by shifting to a more reliable backend for critical operations like FastAPI, where we have full control over logic and state changes.

For the QR scanner, we improved how the scanned data is sent and validated by using local validation techniques that ensure accurate processing and matching of the scanned data with the user's booking information.

3.2 Lessons Learned

The development of *Murir Tin* was a valuable learning experience for our entire team. Through the process of building a smart local bus app with real-time features and a hybrid tech stack, we gained both technical and non-technical insights:

1. **Integration of Multiple Technologies:** Working with Flutter, FastAPI, and Supabase taught us how to manage communication between different platforms. We learned to structure a modular backend, efficiently handle authentication workflows, and bridge frontend-backend interactions using JWT.
2. **Use of Git & Github:** Collaborating via Git and GitHub improved our version control skills and facilitated smoother teamwork. We practiced proper branching, merging, and pull request reviews, which helped minimize conflicts and track changes effectively.
3. **Frontend Development Challenges:** We faced challenges like managing location permissions and fetching data asynchronously. These experiences taught us how important it is to design user-friendly and responsive interfaces that handle errors well.
4. **Backend API Design and Testing:** Building RESTful APIs with FastAPI and integrating them with Supabase improved our skills in database design, authentication, and authorization. Testing the APIs with Swagger helped us ensure everything worked correctly and handled errors properly.
5. **Team Collaboration and Communication:** Coordinating between the frontend, backend, and map teams showed us how vital regular communication is. Especially when working on connected features like ticket booking, complaints, and live bus tracking, staying in sync was crucial.

3.3 Future Plan

The app is currently in its basic version, offering only the essential features needed to support the planned color-coded bus system. While this version covers the core functions like finding routes, booking tickets, and reporting issues, there is still room for improvement. We plan to enhance the user interface to make it more attractive and easier to navigate, especially for users with different levels of tech skills. Along with the UI upgrade, we aim to add voice assistance for better accessibility, especially for elderly or visually impaired users. Bangla language support will also be included to make the app more inclusive and easier to use for

local commuters. In addition, we are working on officially integrating with BRTA(Bangladesh Road Transport Authority) and bus operators to ensure accurate real-time data and reliable service.

Another key improvement involves the Book History section. Right now, it displays all tickets together, which can be confusing. We plan to organize this by separating tickets into Active (upcoming or unused) and Done (used or expired), helping users manage their travel records more easily and efficiently.